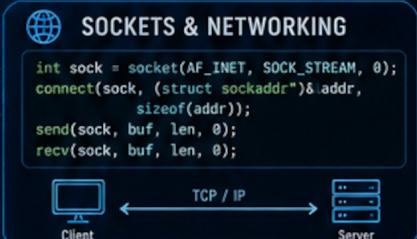
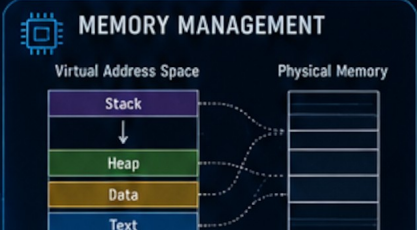
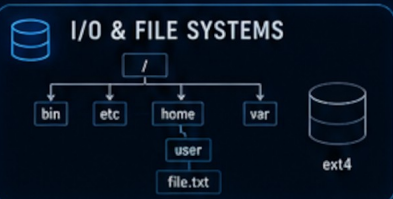
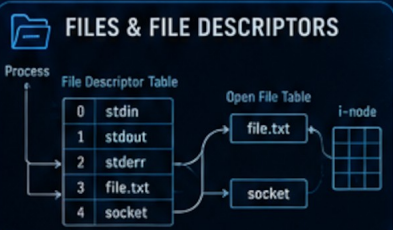
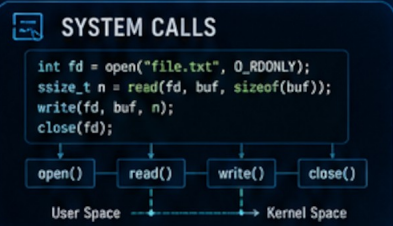
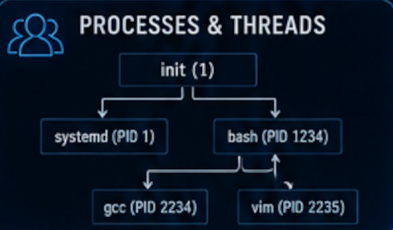


LINUX

SYSTEM PROGRAMMING

THE DEFINITIVE GUIDE

THE COMPLETE RESOURCE FOR LINUX SYSTEM PROGRAMMERS



```
01 #include <stdio.h>
02 #include <unistd.h>
03 #include <sys/types.h>
04 #include <sys/wait.h>
05
06 int main(void) {
07     pid_t pid = fork();
08     if (pid == 0) {
09         execlp("/bin/ls", "ls", "-l", NULL);
10         perror("execlp");
11         _exit(1);
12     } else if (pid > 0) {
13         wait(NULL);
14     }
15 }
```

```
int fd[2];
if (pipe(fd) == -1) {
    perror("pipe");
    return 1;
}
write(fd[1], "hello\n", 6);

char buf[64] = {0};
read(fd[0], buf, sizeof(buf));
printf("%s", buf);
close(fd[0]);
close(fd[1]);
```

```
struct sockaddr_in addr = {0};
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
addr.sin_addr.s_addr =
    inet_addr("127.0.0.1");
int sock = socket(AF_INET, SOCK_STREAM, 0);
connect(sock, (struct sockaddr*)&addr,
    sizeof(addr));
...
close(sock);
```

- ✓ Practical Examples
- ✓ Real-World Projects
- ✓ Best Practices
- ✓ Deep Technical Insights

MARCO BRUNO



Introduction

Linux provides one of the richest and most powerful system programming environments available today. Its interfaces form the foundation of modern operating systems, powering cloud infrastructure, servers, embedded devices, containers, networking platforms, and countless software applications. Yet the Linux programming interface is vast, spanning hundreds of system calls and library functions distributed across numerous standards, manual pages, and reference documents.

The goal of this book is to bring these interfaces together in a single, coherent, and comprehensive reference. Rather than focusing on a particular subsystem or presenting a catalog of isolated APIs, it covers the full spectrum of Linux system programming, from file I/O and filesystems to process management, memory, threads, synchronization, interprocess communication, networking, security, authentication, and terminal programming. The result is a resource that can be read sequentially as a textbook or consulted as a standalone reference by experienced developers.

The book follows the natural organization of the Linux execution environment. It begins with file I/O and the filesystem, then explores the runtime environment, memory management, processes and threads, and finally the mechanisms that enable communication between processes and interaction with users. Throughout the text, POSIX interfaces are presented alongside Linux-specific extensions, highlighting both portability and the capabilities unique to Linux.

Particular emphasis is placed on modern Linux technologies, including *io_uring*, *namespaces*, *capabilities*, *seccomp*, filesystem event monitoring, ELF internals, advanced memory management, POSIX threads, *futexes*, UNIX domain sockets, *systemd services*, and Pluggable Authentication Modules (PAM). These topics are integrated into the broader discussion of Linux programming rather than treated as isolated advanced subjects.

A distinguishing feature of the book is its extensive use of practical examples. Every major interface is accompanied by carefully designed, runnable programs that demonstrate real-world programming techniques, common usage patterns, and best practices. Theory sections complement the examples by explaining not only how an interface works, but also why it exists, how it interacts with other components of the operating system, and when it should be used.

Although no single volume can describe every Linux interface in exhaustive detail, this book aims to provide comprehensive coverage of the system programming facilities that developers encounter when building modern Linux software. Together with the Linux manual pages, it is intended to serve as a complete and practical reference for Linux system programming.

Whether used for structured learning, day-to-day development, or as a desk reference, this book is designed to help readers develop both a practical command of Linux APIs and a deeper understanding of the operating system that supports them.

The source code is available on Github:

<https://github.com/mbgit2026/Linux-System-Programming-The-Definitive-Guide>

Part I – File I/O

This part introduces the Linux I/O model and the interfaces used to read from and write to files and devices. It covers standard and low-level I/O, buffering, file descriptors, asynchronous I/O, file locking, pipes, and techniques for building efficient and reliable I/O-intensive applications.

Part II – Filesystem

This part explores how Linux stores and organizes data. It explains files, directories, permissions, ownership, links, mounts, namespaces, capabilities, quotas, and the kernel interfaces used to manage and monitor filesystems.

Part III – Linux Runtime Environment

This part presents the services that define a process's execution environment. It covers time and locale management, users and groups, authentication, environment variables, system databases, and the runtime facilities that applications use to interact with the operating system.

Part IV – Memory

This part examines how Linux programs are loaded, executed, and managed in memory. It discusses executable files, process startup and termination, memory allocation, memory mapping, protection mechanisms, dynamic loading, and advanced memory management techniques.

Part V – Processes and Threads

This part explains how Linux creates, executes, and coordinates concurrent programs. It covers processes, signals, timers, threads, synchronization primitives, scheduling, security mechanisms, and the development of long-running system services.

Part VI – IPC and Terminals

This part focuses on communication between processes and interaction with users. It introduces POSIX IPC mechanisms, sockets, shared memory, semaphores, UNIX domain sockets, terminal programming, and pseudoterminals used to build distributed and interactive applications.

Conventions used in this book

In this book we will use the following format for the code, the function prototypes and the execution of programs.

Code

The `hello.c` program prints a string on the console.

```
1 #include <stdio.h>

2 int main()
3 {
4     printf("Hello, world! \n");
5     return 0;
6 }
```

hello.c

Function prototype

The `printf`¹ function is defined as follows.

```
#include <stdio.h>

int printf(const char *format, ...);
```

Returns: the number of printed characters

Execution

We can see the manual page of the `printf` function using the `man`² program.

```
[linux@basis] man 3 printf
```

```
[linux@basis] man -s 3 printf
```

```
[linux@basis] man -S 3 printf
```

```
[linux@basis] man --sections=3 printf
```

The options `-S`, `-s` and `-sections` specify the section to which the manual page belong.

1 See the `printf(3)` manual page

2 See the `man(1)` manual page

Code analysis

The code analysis can be presented in three different formats:

1. Line-by-line (italicized)

Line 4: Prints the string to standard output.

Line 5: Exits the program successfully

2. Explanatory

At line 4, the string is printed to standard output using `printf`

At line 5, the program terminates successfully using `return`

3. Highlighted

The call

```
4      printf("Hello, world! \n");
```

prints the message to standard output followed by a newline.

Text conventions

In the text, function names are written without parentheses (). Although this may initially seem unusual in the UNIX and Linux environment, the author intentionally adopted this convention for several reasons. One of the main motivations is to help the reader become familiar with recognizing system calls and library functions immediately by name, an important skill when studying Linux system programming.

Another stylistic choice differs from the common notation typically used in manuals and documentation. Instead of using angle brackets in paths such as `/proc/<pid>/fd`, the text uses the form `/proc/pid/fd`, where *pid* is written in italics to indicate a variable component of the path. This notation was chosen to improve readability and maintain consistency throughout the text.

For each system call and library function discussed in the text, a footnote is included containing a reference to the corresponding manual page. This allows the reader to quickly access the official documentation and explore additional details, usage information, return values, error conditions, and related functions.

Table of contents

Part I - File I/O

I.1 Reading files

- I.1.1 Opening files: `open` and `close`

- I.1.2 Reading from file: `read`

- I.1.3 Writing to file: `write`

- I.1.4 The standard library functions: `fopen`, `fclose` and `fcloseall`

- I.1.5 Reading one character at a time: `fgetc` and `fputc`

- I.1.6 Reading one line at a time: `fgets` and `fputs`

I.2 Opening files

- I.2.1 The `fdopen` and `fileno` functions

- I.2.2 The `freopen` function

- I.2.3 The open flags

I.3 Writing files

I.4 Setting file position

- I.4.1 The `lseek` function

- I.4.2 Getting and setting the position: `fgetpos` and `fsetpos`

- I.4.3 The `fseeko` and `ftello` functions

I.5 Appending to files

I.6 Creating files

- I.6.1 The open `O_EXCL` flag

I.7 Truncating files

I.8 Reading and writing at a given offset

I.9 Duplicating a file descriptor

- I.9.1 The `dup2` function

- I.9.2 The `dup3` function

- I.9.3 Setting the `O_CLOEXEC` flag

- I.9.4 `dup` errors

I.10 Synchronizing the data

- I.10.1 The `sync` function

- I.10.2 Synchronization errors

I.11 File flags

- I.11.1 Duplicating a file descriptor

- I.11.2 File descriptor flags

- I.11.3 File status flags

- I.11.4 File descriptors and open file description

I.12 Multiplexed I/O

- I.12.1 The `poll` function

- I.12.2 The `ppoll` function

- I.12.3 The `epoll` interface

I.13 Binary I/O

I.14 Error handling

- I.14.1 The `errno` macro

- I.14.2 The `strerror*` functions

- I.14.3 The *end-of-file* and error indicators

- I.14.4 The `err` and `warn` functions
 - I.14.5 The *glibc* error reporting functions
 - I.14.6 The `program_invocation_name` variable
- I.15 Buffered I/O
 - I.15.1 Buffering types
 - I.15.2 The interface to `stdio` `FILE` structure
 - I.15.3 The `fwide` function
- I.16 Memory streams
- I.17 Formatted I/O
 - I.17.1 `printf` and `scanf`
 - I.17.2 Other `printf*` functions
- I.18 System calls and library functions
- I.19 Scatter-Gather I/O
 - I.19.1 The `readv` and `writew` functions
 - I.19.2 The `preadv` and `pwritev` functions
- I.20 Unreading
- I.21 Formatted I/O with variable number of arguments
 - I.21.1 `vprintf`
 - I.21.2 `vscanf`
 - I.21.3 `sscanf`
- I.22 Temporary files
 - I.22.1 The `mkstemp*` family functions
 - I.22.2 The `tmpfile` function
 - I.22.3 Temporary files with `open`
 - I.22.4 Temporary directories: `mkdtemp`
- I.23 Advices
- I.24 Asynchronous I/O
 - I.24.1 POSIX AIO
 - I.24.2 Multiple I/O requests in one call
 - I.24.3 The `aio_init` function of the real-time library
 - I.24.4 Linux AIO – `io_uring`
 - I.24.5 The `liburing` library
 - I.24.6 Copying a file with `libring`
 - I.24.7 The `io_uring` system calls
- I.25 File locking
 - I.25.1 Advisory record locking
 - I.25.2 Opening file description locks
 - I.25.3 Locks on the system
 - I.25.4 The `flock` function
 - I.25.5 The `lockf` function
- I.26 Stream locking
 - I.26.1 Locking `FILE` for `stdio`
 - I.26.2 Nonlocking `stdio` functions
 - I.26.3 Nonlocking functions for *wide-characters*
- I.27 Pipes and FIFO
 - I.27.1 The `pipe` function
 - I.27.2 The `popen` function

- I.27.3 Two-way communication
- I.27.4 FIFOs
- I.27.6 Pipes tunable values
- I.27.7 I/O on pipes and FIFOs
- I.27.8 Pipes and formatted I/O
- I.27.9. The `splice` and `vmsplice` functions
- I.27.10 Duplicating pipe data
- I.28 `fcntl` commands
 - I.28.1 Managing signals
 - I.28.2 Leases
 - I.28.3 File and directory change notification: `dnotify`
 - I.28.4 Changing the capacity of a pipe
 - I.28.5 File Sealing
 - I.28.6 File read/write hints
- I.29 File allocation

Part II - Filesystem

II.1 Files

- II.1.1 Stat of files: `stat`, `fstat` and `lstat`
- II.1.2 File permissions
- II.1.3 Process credentials and privileges
- II.1.4 The file mode creation mask: `umask`
- II.1.5 Checking file accessibility: `access` and `faccessat`
- II.1.6 Changing file permissions: `chmod` and `fchmod`
- II.1.7 Changing file ownership: `chown`, `fchown` and `lchown`
- II.1.8 Creating and removing links: `link`, `linkat` and `unlink`
- II.1.9 Renaming files and directories: `rename` and `renameat`
- II.1.10 Symbolic links: `symlink` and `readlink`
- II.1.11 File time management: `utimensat`, `futimens`, `utime`, `futimes`
- II.1.12 I-node flags: `ioctl`
- II.1.13 Immutable files
- II.1.14 Extended attributes: `setxattr`, `getxattr`, `removexattr`, `listxattr`

II.2 Directories

- II.2.1 Creating and removing directories: `mkdir`, `mkdirat` and `rmdir`
- II.2.2 Opening and reading directories: `opendir`, `fdopendir` and `readdir`
- II.2.3 Closing directories: `closedir`
- II.2.4 Scanning a directory: `scandir` and `scandirat`
- II.2.5 Seeking a directory: `tellldir`, `seekdir` and `rewinddir`
- II.2.6 Directory file descriptor: `dirfd`
- II.2.7 File tree walk: `nftw`
- II.2.8 Current working directory: `getcwd`, `chdir` and `fchdir`
- II.2.9 Process root directory: `chroot`

II.3 Pathname and files configuration

- II.3.1 Resolving paths and links: `realpath` and `canonicalize_file_name`
- II.3.2 Working with pathnames: `dirname` and `basename`
- II.3.3 Pathnames and filenames configuration: `fpathconf` and `pathconf`

- II.3.4 Retrieving configuration at runtime with `sysconf`
- II.3.5 Retrieving configuration dependent string variables: `confstr`
- II.3.6 Retrieving configuration from the CLI with `getconf`
- II.3.7 Summary
- II.4 Device files
 - II.4.1 Major and minor numbers: `makedev`, `major` and `minor`
 - II.4.2 Creating a device file: `mknod` and `mknodat`
 - II.4.3 Creating FIFO: `mkfifo` and `mkfifoat`
 - II.4.4 The random number generator: `getrandom` and `getentropy`
- II.5 Access control list
- II.6 Monitoring file events
 - II.6.1 The *inotify* interface
 - II.6.2 The *fanotify* interface
 - II.6.3 Monitoring a directory with `fanotify`
- II.7 Capabilities
 - II.7.1 File capabilities
 - II.7.2 The *Bounding Set*
 - II.7.3 The *Securebits* flags
 - II.7.4 Listing the contents of the capabilities sets
 - II.7.5 Capabilities data types
 - II.7.6 Child capabilities
- II.8 The Linux file system
 - II.8.1 The *ext4* layout
 - II.8.2 Mounting a filesystem: `mount` and `umount`
 - II.8.3 Remounting, binding and changing propagation type
 - II.8.4 Moving a mount and creating a new mount
 - II.8.5 Unmounting a filesystem
 - II.8.6 Journaling and *ext4* filesystem operations
 - II.8.7 The *swap* area: `swapon` and `swapoff`
 - II.8.8 The `proc` file system
 - II.8.9 The `tmpfs` file system
 - II.8.10 The `sysfs` file system
 - II.8.11 Retrieving filesystem statistics : `statfs` and `statvfs`
 - II.8.12 Mount namespaces
 - II.8.13 Linux Namespaces: `unshare`
 - II.8.14 The `setns` function
- II.9 Quotas and resource limits
 - II.9.1 Managing quotas: `quotactl`
 - II.9.2 Resource limits: `getrlimit`, `setrlimit` and `prlimit`

Part III – Linux Runtime Environment

- III.1 Time functions
 - III.1.1 Getting the time: `time`, `gettimeofday` and `settimeofday`
 - III.1.2 Setting time with `clock_gettime` and `clock_settime`
 - III.1.3 Clocks on Linux
 - III.1.4 Setting a new time: `tm`

- III.1.5 Time conversion functions
- III.1.6 Formatting date and time: `strftime` and `strptime`
- III.1.7 Time conversion: `getdate`
- III.2 The environment
 - III.2.1 Working with environment variables: `getenv` and `setenv`
 - III.2.2 Environment during an `execve`
- III.3 Locale
 - III.3.1 Setting locale: `setlocale`
 - III.3.2 Working with locale: `localeconv`, `newlocale` and `uselocale`
 - III.3.3 Duplicating a locale object: `duplocale`
- III.4 The User Database: `/etc/passwd`
 - III.4.1 User and group files
 - III.4.2 The `/etc/passwd` file
 - III.4.3 Iterating the user database : `getpwent` and `setpwent`
 - III.4.4 Working with records : `fgetpwent` and `putpwent`
- III.5 The Group Database: `/etc/group`
 - III.5.1 Reading the group database: `getgrnam` and `getgrgid`
 - III.5.2 Working with the database: `getgrent` and `setgrent`
 - III.5.3 Retrieving the group list: `getgrouplist`
 - III.5.4 Supplementary groups: `getgroups`, `setgroups` and `initgroups`
 - III.5.5 Real and effective group ID: `getgid`, `getegid`, `setgid`, `setegid`
 - III.5.6 Saved Set-User-Id and Set-Group-Id
 - III.5.7 Checking whether a process is in a group: `group_member`
- III.6 Users functions
 - III.6.1 Setting user ID: `getuid`, `geteuid`, `setuid` and `seteuid`
 - III.6.2 `getresgid` and `getresuid`
- III.7 The Password Database: `/etc/shadow`
 - III.7.1 Creating an encrypted passphrase: `crypt`
 - III.7.2 Generating a salt: `crypt_gensalt`
 - III.7.3 Hiding the input with BSD `readpassphrase`
- III.8 Login
 - III.8.1 Working with the `utmp` file: `getutent`, `getutid`, `setutent`
 - III.8.2 The login: `login` and `logout`
- III.9 The host, network, protocol and service databases
 - III.9.1 The host database: `/etc/hosts`
 - III.9.2 The network database: `/etc/networks`
 - III.9.3 The protocol database: `/etc/protocols`
 - III.9.4 The service database: `/etc/services`
- III.10 System identification
 - III.10.1 Getting system information: `uname`
 - III.10.2 Getting and setting hostname: `gethostname` and `sethostent`
- III.11 Pluggable Authentication Modules
 - III.11.1 PAM Authentication: `pam_authenticate`
 - III.11.2 PAM Session: `pam_open_session` and `pam_close_session`
 - III.11.3 Process environment and PAM environment: `pam_getenv`, `pam_putenv`
 - III.11.4 PAM items: `pam_get_item` and `pam_set_item`
 - III.11.5 Changing password: `pam_chauthtok`

- III.11.6 PAM functions summary
- III.12 Insights
 - III.12.1 The Process Execution Environment
 - III.12.2 The `intmax_t` and `uintmax_t` data types
 - III.12.3 Using `initgroups` to set up the supplementary groups
 - III.12.4 Command line tools for the groups
 - III.12.5 The *Name Service Switch*
 - III.12.6 Command line tools for the users
 - III.12.7 The login process

Part IV - Memory

- IV.1 Executable files
 - IV.1.1 The *Executable and Linkable Format* (ELF)
 - IV.1.2 The ELF Header
 - IV.1.3 Sections and Segments
 - IV.1.4 The Symbol Table
 - IV.1.5 The Relocation Table
 - IV.1.6 The Dynamic Section
 - IV.1.7 The Note Section
 - IV.1.8 Reading ELF
 - IV.1.9 Loading ELF into memory
- IV.2 Process Termination
 - IV.2.1 The `main` function
 - IV.2.2 Non-standard process termination: `_exit` and `_Exit`
 - IV.2.3 Normal process termination: `exit`
 - IV.2.4 Exit handlers: `at_exit` and `on_exit`
 - IV.2.5 Process termination and exit status
 - IV.2.6 Aborting a program: `abort`
- IV.3 Command line arguments
 - IV.3.1 Parsing options with `getopt`
 - IV.3.2 Parsing options with `getopt_long`
 - IV.3.3 The `getsubopt` function
 - IV.3.2 The *argp* interface
- IV.4 Memory allocation
 - IV.4.1 Memory allocation functions
 - IV.4.2 Processes and memory
 - IV.4.3 The GNU Allocator
 - IV.4.4 Memory reclamation using `malloc_trim`
 - IV.4.5 Automatic stack allocation with `alloca`
 - IV.4.6 Aligned memory allocation with `posix_memalign`
 - IV.4.7 Retrieving the system page size with `getpagesize`
- IV.5 Memory mapped I/O
 - IV.5.1 Memory mapping with `mmap`
 - IV.5.2 Private file mapping
 - IV.5.3 Shared file mapping
 - IV.5.4 Anonymous mapping
 - IV.5.5 Extending a mapped file

- IV.5.6 Memory remapping with `mremap`
- IV.6 Memory functions
 - IV.6.1 Memory copying with `memcpy`
 - IV.6.2 Safe memory copying with `memmove`
 - IV.6.3 Memory initialization with `memset` and `bzero`
 - IV.6.4 Memory searching with `memchr` and `memmem`
 - IV.6.5 Memory comparison with `memcmp`
 - IV.6.6 Memory obfuscation with `memfrob`
- IV.7 Protecting the memory
 - IV.7.1 Memory protection with `mprotect`
 - IV.7.2 Memory protection keys
- IV.8 Locking memory
 - IV.8.1 Memory locking with `mlock`
 - IV.8.2 Memory residency inspection with `mincore`
- IV.9 RAM-Backed Files and Secret Memory
 - IV.9.1 Anonymous RAM-backed files: `memfd_create`
 - IV.9.2 Secret memory regions with `memfd_secret`
- IV.10 Memory advice
 - IV.10.1 Memory advice with `madvise`
 - IV.10.2 Portable memory advice with `posix_madvise`
 - IV.10.3 Remote memory advice with `process_madvise`
- IV.11 Nonlocal gotos
 - IV.11.1 The `setjmp` and `longjmp` functions
 - IV.11.2 The `sigsetjmp` and `siglongjmp` functions
- IV.12 Core dump file
- IV.13 Dynamically loaded objects
 - IV.13.1 Opening a shared object: `dlopen`
 - IV.13.2 Obtaining the address of a symbol: `dlsym`
 - IV.13.4 Obtaining information about a loaded object: `dlinfo`
 - IV.13.5 Translating address to symbolic information: `dladdr`
 - IV.13.6 Walking through a list of shared objects: `dl_iterate_phdr`

Part V – Processes and Threads

- V.1 Processes
 - V.1.1 Getting the process identifier: `getpid` and `getppid`
 - V.1.2 Creating a process: `fork`
 - V.1.3 Creating a process blocking the parent: `vfork`
 - V.1.4 Blocking execution: `wait`, `waitpid` and `waitid`
 - V.1.5 The `wait3` and `wait4` functions
 - V.1.6 Process file descriptors
 - V.1.7 Creating child processes: `clone` and `clone3`
 - V.1.8 Comparing processes: `kcmp`
 - V.1.9 Executing a shell command: `system`
 - V.1.10 Executing a program: `execve`
 - V.1.11 The `fork` + `execve` pattern
 - V.1.12 `execve` and program arguments

- V.1.13 Process attributes through `execve`
- V.1.14 Dropping privileges before `execve`
- V.1.15 Executing a program via file descriptor: `fexecve`
- V.1.16 Executing a program via memory file descriptor using `fexecve`
- V.1.17 The `exec` family functions
- V.1.18 Controlling a process: `prctl`
- V.1.19 Process data transfer: `process_vm_read`
- V.1.20 Secure Computing state of the process: `seccomp`
- V.1.21 Secure Computing state of the process: `libseccomp`
- V.1.22 Seccomp user-space notification mechanism: `seccomp_unotify`
- V.1.23 Seccomp user-space notification mechanism with `libseccomp`
- V.1.24 Process accounting: `acct`
- V.1.25 Process scheduling: `nice` and `setpriority`

V.2 Process groups, sessions and terminal

- V.2.1 Process groups: `getpgid` and `setpgid`
- V.2.2 Sessions: `setsid` and `getsid`
- V.2.3 Controlling terminal: `tcsetpgrp`
- V.2.4 Job control
- V.2.5 Identifying the controlling terminal: `ctermid` and `ttyname`
- V.2.6 Orphaned process groups

V.3 Signals

- V.3.1 Signal overview
- V.3.2 Signal details
- V.3.3 Describing signals: `strsignal` and `psignal`
- V.3.4 Signal handling: `sigaction`
- V.3.5 Signal set operations: `sigemptyset` and `sigfillset`
- V.3.6 Signal handlers
- V.3.7 Sending signals to the caller: `raise`
- V.3.8 Sending signals to another process: `kill` and `killpg`
- V.3.9 Sending signals to a specific thread: `tgkill` and `pthread_kill`
- V.3.10 Sending real-time signals: `sigqueue` and `pthread_sigqueue`
- V.3.11 Setting alarm clock: `alarm`
- V.3.12 Blocking signals: `sigprocmask`
- V.3.13 Checking for pending signals: `sigpending`
- V.3.14 Handling signals with nonlocal jumps
- V.3.15 Waiting for signals: `pause` and `sigsuspend`
- V.3.16 Waiting for signals: `sigwait` and `sigwaitinfo`
- V.3.17 Signal file descriptor: `signalfd`
- V.3.18 Alternate signal stack: `sigaltstack`
- V.3.19 Sleeping: `sleep`, `nanosleep`, `usleep`, `clock_nanosleep`

V.4 Timers

- V.4.1 Interval timers: `getitimer` and `setitimer`
- V.4.2 POSIX timers: `timer_create`
- V.4.3 Arming and disarming timers: `timer_gettime` and `timer_settime`
- V.4.4 Overruns and deletion: `timer_getoverrun` and `timer_delete`
- V.4.5 CPU-time clocks: `clock_getcpuclockid`, `pthread_getcpuclockid`

V.5 Threads

V.5.1 POSIX threads: `pthread`

V.5.2 Thread ID: `gettid`, `pthread_self` and `pthread_equal`

V.5.3 Creating threads: `pthread_create` and `pthread_join`

V.5.4 Thread attributes: `pthread_attr_init` and `pthread_attr_destroy`

V.5.5 Terminating threads: `pthread_exit` and `pthread_cancel`

V.5.6 Clean-up handlers: `pthread_cleanup_push`, `pthread_cleanup_pop`

V.5.7 Sending signal to threads: `pthread_kill` and `pthread_sigmask`

V.5.8 Thread-specific data: `pthread_key_create`

V.6 Synchronization

V.6.1 Mutexes

V.6.2 Dual-mutex pattern for thread-safe reference-counted objects

V.6.3 Deadlock

V.6.4 Mutex attributes

V.6.5 Timed mutex locking: `pthread_mutex_timedlock`

V.6.6 Timed locking with robust mutexes

V.6.7 Read-Write Locks

V.6.8 Condition variables

V.6.9 Spin locks

V.6.10 Barriers

V.6.11 Fork handlers: `pthread_atfork`

V.6.12 Semaphores

V.6.13 Futexes

V.6.14 Thread name: `pthread_setname_np`

V.7 Daemon processes and logging

V.7.1 Daemon overview

V.7.2 The `daemon` function

V.7.3 The `systemd` service API: `sd-daemon.h`

V.7.4 Logging: `syslog`

V.7.5 Logging: `sd-journal`

Part VI – IPC and Terminals

VI.1 Introduction

VI.2 POSIX Message Queues

VI.2.1 Creating a message queue: `mq_open`, `mq_close`, `mq_unlink`

VI.2.2 Sending and receiving messages: `mq_send` and `mq_receive`

VI.2.3 Message queue attributes: `mq_getattr` and `mq_setattr`

VI.2.4 Message queue notification: `mq_notify`

VI.2.5 Notification via thread

VI.2.6 Notification via signal

VI.3 POSIX Named Semaphores

VI.3.1 Creating and deleting semaphores: `sem_open`, `sem_close`, `sem_unlink`

VI.3.2 Process-local mutual exclusion

VI.3.3 Interprocess mutual exclusion

VI.3.4 Two-way synchronization pattern

VI.4 POSIX Shared Memory

VI.4.1 Overview

VI.4.2 Producer-consumer pattern

VI.4.3 Client-server pattern

VI.5 Sockets

VI.5.1 Overview

VI.5.2 Creating sockets: `socket` and `shutdown`

VI.5.3 Byte order conversion functions

VI.5.4 Addressing: `getaddrinfo` and `getnameinfo`

VI.5.5 Address formats: `inet_ntop` and `inet_pton`

VI.5.6 Setting the socket address: `bind`

VI.5.7 Receiving connections: `listen` and `accept`

VI.5.8 Connecting to sockets: `connect`

VI.5.9 Socket information: `getsockname`, `getpeername`, `getsockopt`

VI.5.10 Data transmission: `send` and `recv`

VI.5.11 Connection-oriented client-server uptime service

VI.5.12 Non-blocking sockets and asynchronous I/O

VI.5.13 Out-of-band data: `socketatmark`

VI.5.14 Sending and receiving multiple messages: `sendmmsg` and `recvmsg`

VI.6 UNIX Domain Sockets

VI.6.1 Unnamed UNIX domain sockets: `socketpair`

VI.6.2 Named UNIX domain sockets: `sockaddr_un`

VI.6.3 Datagram sockets: `SOCK_DGRAM`

VI.6.4 Stream-oriented sockets and message-oriented sockets

VI.6.5 Abstract sockets

VI.6.6 Socket options and ancillary messages: `cmsg`

VI.6.7 Passing file descriptors within a single process

VI.6.8 Passing file descriptors between parent and child processes

VI.6.9 Passing file descriptors and credentials between unrelated processes

VI.6.10 A secure file access service

VI.6.11 Sequenced-packet sockets: `SOCK_SEQPACKET`

VI.7 Terminal I/O

VI.7.1 Identifying terminal file descriptors: `isatty`

VI.7.2 Terminal properties: `termios`

VI.7.3 Retrieving and changing terminal settings: `tcgetattr` and `tcsetattr`

VI.7.4 Canonical, noncanonical and raw mode

VI.7.5 Line control

VI.7.6 Line speed

VI.7.7 Special characters

VI.7.8 Window terminal size

VI.7.9 Command-line utilities

VI.7.10 `ncurses`

VI.8 Pseudo terminals

VI.8.1 Overview

VI.8.2 UNIX 98 pseudoterminal

VI.8.3 `script` and `expect`

VI.8.3 Opening pseudoterminal: `getpt` and `posix_openpt`

VI.8.4 Unlocking pseudoterminal: `grantpt` and `unlockpt`

VI.8.5 Getting pseudoterminal name: `ptsname`

VI.8.6 Pseudoterminal pair: `openpty`, `forkpty` and `loginpty`

Part I

File I/O

I.1. Reading files

In Linux, almost everything is represented as a file: the keyboard, hard disks, terminals, printers, and even communication devices are exposed through the filesystem interface. Regular files are also represented in the same way. Files can be opened to perform read and/or write operations, and each file has associated attributes such as size, access permissions, ownership, and type.

Like most Unix-like operating systems, Linux defines three standard streams as the primary input/output interfaces for programs: *standard input*, *standard output*, and *standard error*. In Linux, these streams are represented as file descriptors and are identified by both a name and a numeric value.

stdin	0	Standard input
stdout	1	Standard output
stderr	2	Standard error

Tab 1. The main I/O devices

When we execute a command from the shell, its output is normally displayed on the *standard output*, while error messages are sent to the *standard error* stream. *Standard input* represents the source from which data is read, typically the keyboard. *Standard output* may correspond to the terminal, a printer, or a file used to store the results of program execution. Likewise, *standard input* and *standard error* can also be redirected to files or other devices instead of the console.

I.1.1 Opening files: `open` and `close`

The `open`³ function opens the file specified by its first argument according to the mode indicated by the second argument, and returns a *file descriptor*. A file descriptor is a positive integer used by the operating system to identify an open file within a program. Every file opened by a process in the system is associated with a file descriptor.

```
#include <fcntl.h>

int open(const char *pathname, int flags);

Returns: the new file descriptor, -1 on error
```

The `openfile.c` program provides an example of how this works.

```
1 #include <stdio.h>
2 #include <fcntl.h>

3 int main (int argc, char *argv[])
4 {
5     int fd;
```

³ See the `open(2)` manual page

```

6     int fd2;
7     int fd3;

8     fd = open("file.txt", O_RDONLY);
9     fd2 = open("file2.txt", O_WRONLY);
10    fd3 = open("file3.txt", O_RDWR);
11    printf("fd = %d", fd);
12    printf("\nfd2 = %d", fd2);
13    printf("\nfd3 = %d", fd3);
14    return 0;
15 }

```

openfile.c

The program opens three files using `open`, each with a different file opening mode, and then prints the file descriptor values returned by the `open` calls.

We create two files and then run the program.

```

[linux@io] touch file.txt file2.txt
[linux@io] ./openfile
fd = 3
fd2 = 4
fd3 = -1

```

The value of the third descriptor is `-1` because the file we attempted to open does not exist. The first descriptor has the value `3` because the descriptors `0`, `1`, and `2` are already reserved for `stdin`, `stdout`, and `stderr`, respectively. For each additional file opened, the operating system typically assigns the next available file descriptor. Therefore, if `file3.txt` existed and were opened successfully, its descriptor would likely be `5`, and so on.

The values the `flags` argument may have, are listed in the following table.

<code>O_RDONLY</code>	Read only
<code>O_WRONLY</code>	Write only
<code>O_RDWR</code>	Read/Write
<code>O_APPEND</code>	Append

Tab 2. Opening modes (`open`)

In the previous program, we did not close the files, even though closing files is a fundamental operation when working with them. Although all open file descriptors are automatically closed when the program terminates, it is good practice to close them explicitly to ensure correctness and avoid wasting system resources.

The `close`⁴ function closes the file associated with the file descriptor passed as an argument.

⁴ See the `close(2)` manual page

```
#include <unistd.h>

int close(int fd);
```

Returns: 0 on success, -1 on error

The `closefile.c` program shows an example.

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>

4 int main (int argc, char *argv[])
5 {
6     int fd;
7     fd = open("file.txt", O_RDONLY);
8     close(fd);
9     return 0;
10 }
```

closefile.c

After opening the file in read-mode we close it using the `close` function.

We execute the program

```
[linux@io] ./closefile
[linux@io]
```

The program opens and then closes the file.

I.1.2 Reading from file: read

When we want to read the file contents we can use the `read`⁵ function, which reads `count` characters from the `fd` file descriptor and stores them in `buf`.

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);
```

Returns: the number of bytes read, -1 on error

`read` returns the number of bytes actually read. In some cases, this value may be smaller than the number of bytes requested, for example, when the *end of file* is reached before the requested amount has been read, when reading from a terminal device, a network device, or a pipe, or when the

5 See the `read(2)` manual page

operation is interrupted by a signal. For this reason, it is always best practice to check the returned value to detect a partial read or an error.

After a successful call, the *file offset* is automatically increased by the number of bytes read.

The `readfromfile.c` program reads from a file.

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>

4 #define COUNT 128
5 int main (int argc, char *argv[])
6 {
7     int fd;
8     ssize_t n;
9     char buffer[COUNT];

10    fd = open("./file.txt", O_RDONLY);
11    if (fd < 0) {
12        printf("Open error - exit..");
13        return 1;
14    }

15    n = read(fd, buffer, COUNT - 1);
16    if (n < 0) {
17        printf("Read error\n");
18        close(fd);
19        return 1;
20    }

21    buffer[n] = '\0';
22    printf("Bytes read: %d", n);
23    printf("\nContent: %s", buffer);
24    close(fd);
25    return 0;
26 }
```

readfromfile.c

We use a 128-byte buffer to store the data. We call `read` at line 15 passing the file descriptor, the buffer and `COUNT - 1`. By reading at most `COUNT - 1` bytes, we leave space for the terminating `'\0'`, making `buffer` a valid C string.

The `ssize_t`⁶ data type is designed to store the return value of functions such as `read`, including error indications.

At line 16, we check the value returned by `read`. If an error occurred, we print an error message and exit with a status code of 1.

At line 21, we append a *null terminator* to the buffer so that it can be safely treated as a C string and printed using `%s`.

6 See the `size_t(3type)` manual page

We write some data into the `file.txt` file and then run the program.

```
[linux@io] echo Linux > file.txt
[linux@io] ./readfromfile
Bytes read: 6
Content: Linux
```

The buffer size is 128 bytes, but the file contains only 6 bytes. The `read` function reads data from the file up to the specified buffer size (128 bytes). This means it will return up to 128 bytes if available; however, since the file is smaller, only 6 bytes are actually read, and no additional data is returned beyond the end of the file.

I.1.3 Writing to file: `write`

The `write`⁷ function is used to write data to a file descriptor. It writes up to `count` bytes from the buffer `buf` to the file descriptor `fd` and returns the number of bytes actually written.

```
#include <unistd.h>

ssize_t write(int fd, const void *buf, size_t count);

Returns: the number of written bytes, -1 on error
```

For the *standard input*, *standard output* and *standard error*, the corresponding constants are defined in the `unistd.h` header file. These constants refer to their respective file descriptors.

STDIN_FILENO	0
STDOUT_FILENO	1
STDERR_FILENO	2

Tab 3. Standard descriptors

The `readfile.c` program opens a file, repeatedly reads from it until at least one byte is obtained, prints the number of bytes read along with the data, and then closes the file.

```
1 #include <stdio.h>
2 #include <fcntl.h>
3 #include <unistd.h>

4 #define COUNT 128

5 int main()
6 {
7     int fd;
8     ssize_t n;
```

⁷ See the `write(2)` manual page

```

9     char buffer[256];

10    fd = open ("../file.txt", O_RDONLY);
11    if (fd < 0) {
12        printf ("Error opening file - exit..");
13        return 1;
14    }

15    while ((n = read(fd, buffer, COUNT)) > 0) {
16        if (write(STDOUT_FILENO, buffer, n) != n) {
17            printf("Write error\n");
18            close(fd);
19            return 1;
20        }
21    }
22    if (n < 0) {
23        printf("Read error\n");
24        close(fd);
25        return 1;
26    }
27    close(fd);
28    return 0;
29 }

```

readfile.c

At lines 3–5, we declare three variables: `fd`, which stores the file descriptor returned by `open`; `n`, declared as `ssize_t`, which is the appropriate data type for storing the return values of `read` and `write`; and `buffer`, a 256-byte array used to temporarily store the data read from the file. At line 7, we open `file.txt` in read-only mode using `open`. If `open` returns a negative value, an error has occurred. In that case, we print an error message and terminate the program by returning 1.

At line 13, we call `read` inside the `while` loop to read up to `COUNT` bytes from the file. The number of bytes actually read is stored in `n`, and the loop continues as long as `read` returns a positive value. At line 14, we call `write` to write exactly `n` bytes to the standard output (`STDOUT_FILENO`). Using `n` instead of `COUNT` is important because the last call to `read` may return fewer bytes than requested. At line 14, we immediately check the value returned by `write`. If it differs from `n`, either a *partial write* or an error has occurred. In that case, we print an error message, close the file descriptor, and terminate the program. At line 20, after the loop terminates, we check whether `n` is negative. If so, `read` failed, so we print an error message, close the file descriptor, and exit with a status code of 1. The `while` loop can terminate for two reasons: either `read` returns 0, indicating that the end of the file has been reached, or it returns `-1`, indicating an error. The check at line 20 allows us to distinguish between these two cases. At line 26, we close the file descriptor to release the associated system resource. Finally, we return 0 to indicate that the program completed successfully.

We execute the program

```

[linux@io] ./readfile
Linux

```

Unlike the `readfromfile.c` program, the `readfile.c` program reads from the file until data is available. Each call to `read` retrieves up to 128 bytes from the file. Let's run both programs using a file larger than 128 bytes.

```
[linux@io] cp /etc/passwd file.txt
[linux@io] ./readfromfile
Bytes read: 128
Content: io:1000:1000::/home/io:/usr/bin/zsh
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

[linux@io] ./readfile
io:1000:1000::/home/io:/usr/bin/zsh
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
...output...
```

The `readfromfile.c` program reads 128 bytes from the file, while the `readfile.c` program continues reading the entire file in chunks of 128 bytes (the size of the buffer). The size of `buf` can be greater than `count`, but `write` will still write only `count` bytes. The data type `ssize_t`⁸ is used for counting bytes and is defined in the `sys/types.h` and `unistd.h` header files.

I.1.4 The standard library functions: `fopen`, `fclose` and `fcloseall`

The `open`, `read`, `close`, and `write` functions are *system calls*. The *standard library* provides a set of file I/O functions that allow files to be handled as *streams*. These functions are not Linux system calls; they belong to the *C standard library* and are defined in the `stdio.h` header file.

The `fopen`⁹ function is used to open a file and associate it with a *stream*. It takes as arguments the file name and the opening mode, and returns a pointer to the corresponding *file stream*. The `fclose`¹⁰ function closes the file associated with the stream passed as its argument.

```
#include <stdio.h>

FILE *fopen(const char *pathname, const char *mode);

                                Return: a pointer to the file, NULL on error

int fclose(FILE *stream);

                                Returns: 0 on success, EOF on error
```

When working with multiple open files, it may be useful to close all streams at once. The `fcloseall`¹¹ function closes all streams opened by the process.

8 See the `system_data_types(7)` manual page

9 See the `fopen(3)` manual page

10 See the `fclose(3)` manual page

11 See the `fcloseall(3)` manual page

```
#include <stdio.h>

int fcloseall(void);
```

Returns: 0 on success, EOF on error

I.1.5 Reading one character at a time: `fgetc` and `fputc`

The `fgetc`¹² and `fputc`¹³ functions are used to read and write a single character, respectively. The first reads a character from the specified input stream and returns it. The second writes a character to the specified output stream and returns the written character.

```
#include <stdio.h>

int fgetc(FILE *stream);

int fputc(int c, FILE *stream);
```

Returns: the read character, EOF on error

Return: the written character, EOF on error

The `readfilef.c` program reads the contents of the file and prints them to the screen.

```
1 #include <stdio.h>

2 int main()
3 {
4     FILE *file;
5     int c;

6     file = fopen("./file.txt", "r");
7     while (c!=EOF) {
8         c=fgetc(file);
9         if(c!=EOF)
10             fputc(c, stdout);
11     }
12     fclose(file);
13     return 0;
14 }
```

readfilef.c

The `while` loop continues iterating until `c` becomes `EOF` (End of File). In each iteration, it reads one byte from the file using `fgetc`, and if the byte read is valid, it prints it to `stdout`.

12 See the `fgetc(3)` manual page

13 See the `puts(3)` manual page

file.txt

```
Linux Programming
1234567890
```

The program is executed

```
[linux@io] ./readfilef
Linux Programming
1234567890
```

The `FILE` data type is an object used to represent streams, and it is defined in the `stdio.h` header file. The standard library also defines constants in `stdio.h` for standard input, standard output, standard error, and end-of-file handling.

<code>stdin</code>	0
<code>stdout</code>	1
<code>stderr</code>	2
<code>EOF</code>	-1

Tab 4. The standard streams

We present the same program using a more concise syntax.

```
1 #include <stdio.h>

2 int main()
3 {
4     int c;
5     FILE *fp;
6     fp = fopen("./file.txt", "r");
7     while ((c=fgetc(fp)) != EOF)
8         fputc(c, stdout);
9     fclose(fp);
10    return 0;
11 }
```

readfilefs.c

```
[linux@io] ./readfilefs
Linux Programming
1234567890
```

The `fopen` function accepts the following flags:

r	Read
r+	Read and Write
w	Write. If the file exists its contents are truncated to zero
w+	Read and Write. If the file does not exist it is created
a	Writes at the end of the file (appends to existing content)
a+	Reading and queuing. If the file does not exist it is created

Tab 5. File opening modes of `fopen`

In addition to the standard flags `fopen` also accepts the following

c	The file opens with deletion disabled
e	The descriptor will be closed if one of the <code>exec</code> ¹⁴ functions is used (corresponds to the <code>FD_CLOEXEC</code>)
m	The file is opened and accessed via <code>mmap</code> ¹⁵
x	Open the file exclusively

Tab 6. `fopen` flags

The correspondence between the `fopen` and `open` flags is shown in the following table.

fopen	open
r	O_RDONLY
w	O_WRONLY O_CREAT O_TRUNC
a	O_WRONLY O_CREAT O_APPEND
r+	O_RDWR
w+	O_RDWR O_CREAT O_TRUNC
a+	O_RDWR O_CREAT O_APPEND

Tab 7. File opening methods (`open` and `fopen`)

The `stdin2stdout.c` program copies data from standard input to the standard output, one character at a time.

```

1 #include <stdio.h>

2 int main()
3 {
```

14 See the `exec(3)` manual page

15 See the `mmap(2)` manual page

```
4     int c;

5     while( (c=fgetc(stdin)) != EOF)
6         fputc(c, stdout);
7     return 0;
8 }
```

stdin2stdout.c

The program reads one character at a time from standard input (`stdin`) and prints it to standard output (`stdout`), effectively echoing the characters typed by the user onto the screen.

```
[linux@io] ./stdin2stdout
hello
hello
Linux
Linux
```

I.1.6 Reading one line at a time: `fgets` and `fputs`

The `fgets`¹⁶ function reads a line from the file passed as an argument and stores it in a buffer. It takes as parameters the buffer where the line will be stored, the maximum number of bytes to read, and the file stream from which to read, and it returns the resulting string.

The `fputs`¹⁷ function writes a line to the file passed as an argument. It takes as parameters the buffer containing the data to be written and the output file stream.

```
#include <stdio.h>

char *fgets(char *s, int size, FILE *stream);

                                Return: the buffer read, NULL on error

int fputs(const char *s, FILE *stream);

                                Return: a non negative integer, EOF on error
```

The `readfileline.c` program uses `fgets` to read the file one line at a time.

```
1 #include <stdio.h>

2 int main()
3 {
4     FILE *file;
5     char buffer[1024];
6     int count = 1024;
```

16 See the `fgetc(3)` manual page

17 See the `puts(3)` manual page

```
7     if ((file = fopen("./file.txt", "r")) == NULL) {
8         printf ("Error opening file");
9         return 1;
10    }
11    while (fgets (buffer, count, file))
12        fputs (buffer, stdout);

13    fclose (file);
14    return 0;
15 }
```

readfileline.c

This version of the program uses `fopen` to open the file, `fgets` to read from the stream, and `fputs` to output the read data to the screen.

```
[linux@io] ./readfileline
Hello world

1234567890
```

The program reads the file one line at a time and prints it to the screen. The `fgets` function attempts to read up to `count - 1` bytes from the stream; if it encounters an end-of-file (EOF) or a newline character, it stops reading.

The `stdin2stdoutline.c` program copies standard input to standard output, line by line.

```
1 #include <stdio.h>

2 int main()
3 {
4     char buf[256];
5     while(fgets(buf, 256, stdin) != NULL)
6         fputs(buf, stdout);
7 }
```

stdin2stdoutline.c

The `fgets` call attempts to read up to 256 bytes from standard input until it encounters an EOF or a newline character, and stores the data in the buffer `buf`. `fputs` writes the contents of `buf` to standard output.

```
[linux@io] ./stdin2stdoutline
hello Linux
hello Linux
```

The program copies each input line to standard output.

I.2. Opening files

To open files, the `open`¹⁸ function internally invokes the `openat` system call.

```
#include <fcntl.h>

int openat(int dirfd, const char *pathname, int flags);

Returns: file descriptor of the open file, -1 on error
```

The `openat` system call behaves like `open`, except when the `pathname` is relative. In that case, it is interpreted relative to the directory referenced by the `dirfd` file descriptor, rather than the current working directory of the calling process.

If the `pathname` is relative and `dirfd` has the special value `AT_FDCWD`, then the `pathname` is interpreted relative to the process's current working directory.

The `open.c` program opens a file and then waits 20 seconds.

```
1 #include <fcntl.h>
2 #include <unistd.h>

3 int main ()
4 {
5     openat(AT_FDCWD, "./file.txt", O_RDONLY);
6     sleep(20);
7 }
```

open.c

The file `file.txt` is opened by resolving its relative path from the current directory. The `sleep` call causes the program to wait for 20 seconds.

We execute the program using `strace`¹⁹

```
[linux@io] touch file.txt
[linux@io] strace ./open
...output...
openat(AT_FDCWD, "file.txt", O_RDONLY) = 3
...output...
```

`openat` returned the file descriptor 3. We use `lsuf`²⁰ to inspect the open file.

```
[linux@io] lsuf | grep file.txt
open  9200          io  3r  REG      259,2      5 11279432 /home/io/file.txt
```

¹⁸ See the `open(2)` manual page

¹⁹ See the `strace(1)` manual page

²⁰ See the `lsuf(8)` manual page

The output of `ls -l` shows the following columns: process name, PID (process ID), user, descriptor (`r` = read), file type, device, file size, inode, filename.

The `openat` function, unlike `open`, resolves a relative pathname starting from the directory referenced by the `dirfd` parameter.

We replace line 4 with the following two lines

```
int fd = open("/home/io/", O_RDONLY);
openat(fd, "file.txt", O_RDONLY);
```

```
[linux@io] strace ./open
...output...
openat(AT_FDCWD, "/home/io/", O_RDONLY) = 3
openat(3, "file.txt", O_RDONLY)      = 4
...output...
```

The program first opens the directory `/home/io` and then uses the returned file descriptor (3) as the base for resolving the path to the file `file.txt`. The second `openat` call returns file descriptor 4.

We replace the previous lines with the following

```
openat(0, "/home/io/file.txt", O_RDONLY);
```

```
[linux@io] strace ./open
...output...
openat(0, "/home/io/file.txt", O_RDONLY) = 3
clock_nanosleep(CLOCK_REALTIME, 0, {tv_sec=50, tv_nsec=0},
...output...
```

`openat` ignores the `dirfd` parameter because the pathname is *absolute* and resolves it directly. The system call `clock_nanosleep`²¹ then pauses execution for 500 seconds before the program finishes.

When using system calls or library functions, it is good practice to check their return values and handle errors appropriately. For example, `open` returns `-1` on failure.

When a system call fails, Linux sets the global variable `errno` to indicate the type of error. Therefore, we can check `errno` and use it to print or interpret the corresponding error message.

The `perror`²² function prints a descriptive error message for the most recent error, using the string passed as its argument as a prefix.

The `openfilecheck.c` program opens a file and checks whether `open` returns an error.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <fcntl.h>
```

²¹ See the `clock_nanosleep(2)` manual page

²² See the `perror(3)` manual page

```

4 int main() {
5     int fd;
6     fd = open("./newfile.txt", O_RDONLY);
7     if (fd == -1)
8         perror("open");
9 }

```

openfilecheck.c

The line 8 prints the error returned by `open`.

```

[linux@io] ./openfilecheck
open: No such file or directory

```

`open` attempts to open the file in read-only mode, but since the file does not exist, it returns the error `ENOENT` (No such file or directory). The same behavior occurs if we try to open the file using the `O_WRONLY` or `O_RDWR` flags.

To create the file if it does not already exist, we can use the `O_CREAT` flag and specify the permissions with which the file should be created. These permissions are listed in the following table.

<code>S_IRWXU</code>	00700	the owner has read, write and execution permissions
<code>S_IRUSR</code>	00400	the owner has read permission
<code>S_IWUSR</code>	00200	the owner has write permission
<code>S_IXUSR</code>	00100	the owner has execution permission
<code>S_IRWXG</code>	00070	the group has read, write and execution permissions
<code>S_IRGRP</code>	00040	the group has read permission
<code>S_IWGRP</code>	00020	the group has write permission
<code>S_IXGRP</code>	00010	the group has execution permission
<code>S_IRWXO</code>	00007	the others have read, write and execution permissions
<code>S_IROTH</code>	00004	the others have read permission
<code>S_IWOTH</code>	00002	the others have write permission
<code>S_IXOTH</code>	00001	the others have execution permission

Tab 8. File permissions flags

In addition to the standard permissions, Linux supports the following

<code>S_ISUID</code>	0004000	set-user-ID bit
<code>S_ISGID</code>	0002000	set-group-ID bit
<code>S_ISVTX</code>	0001000	sticky bit

Tab 9. Special permissions

We modify the program by replacing line 6 with the following:

```
fd = open("./newfile.txt", O_RDONLY | O_CREAT, S_IRWXU);
```

```
[linux@io] ./openfilecheck  
[linux@io] ls -l newfile.txt  
-rwx----- 1 io io 0 13 feb 17.47 newfile.txt
```

The file has been created with read, write, and execute permissions for the owner, and it has been opened in read-only mode.

If we run the program again, the `O_CREAT` flag checks whether the file already exists. If it does, the file is not created again and the existing file is not overwritten. The `O_CREAT` flag only creates the file if it does not already exist.

```
[linux@io] rm newfile.txt  
[linux@io] ./openfilecheck  
[linux@io] ls -l newfile.txt  
-rwx----- 1 io io 0 13 feb 17.47 newfile.txt  
[linux@io] echo "hello" >> newfile.txt  
[linux@io] cat newfile.txt  
hello  
[linux@io] ./openfilecheck  
[linux@io] cat newfile.txt  
hello
```

The content written to the file after the first execution is still present after the second execution. This indicates that `open` did not recreate the file or overwrite it with a new empty one. Additionally, `open` did not return any error indicating that the file already existed.

The `O_EXCL` flag causes `open` to return an error if the file already exists. We replace line 6 with the following.

```
fd = open("./newfile.txt", O_RDONLY | O_CREAT | O_EXCL, S_IRWXU);
```

```
[linux@io] ./openfilecheck  
open: File exists
```

Now, `open` returns the error `EEXIST` (file exists). We replace again the line:

```
fd = open("./newfile.txt", O_RDONLY | O_CREAT | O_EXCL, S_IRWXU);
```

with the previous one

```
fd = open("./newfile.txt", O_RDONLY);
```

If we remove the read permissions from the file and then try to open it, `open` returns different error.

```
[linux@io] chmod u-r newfile.txt  
[linux@io] ls -l newfile.txt
```

```
--wx----- 1 io io 0 13 feb 18.16 newfile.txt
[linux@io] ./openfilecheck
open: Permission denied
```

`open` returns the `EACCESS` (permission denied) error.

The `opfiles.c` program opens two files: the first in read mode, and the second in write mode. The second file is truncated to zero length if it already exists, or created if it does not exist.

```
1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <stdio.h>

4 int main(int argc, char *argv[])
5 {
6     int rfd, wfd;
7     int rflags, wflags, perm;
8     rflags = O_RDONLY;
9     wflags = O_CREAT | O_WRONLY | O_TRUNC;
10    perm = S_IRUSR | S_IWUSR;

11    rfd = open("file.txt", rflags);
12    if (rfd == -1)
13        perror("open");
14    else
15        printf("\nfile.txt open in read mode");

16    wfd = open("file2.txt", wflags, perm);
17    if (wfd == -1)
18        perror("open");
19    else
20        printf("\nfile2.txt open/created in write mode (truncate to 0)");
21    return 0;
22 }
```

opfiles.c

In this program, we check whether `open` succeeds or fails by examining the returned file descriptor value. If an error occurs, we use `perror` to print a descriptive error message.

```
[linux@io] ./opfiles

file.txt open in read mode
file2.txt open/created in write mode (truncate to 0)
```

We write some content into the `file2.txt` file and then re-execute the program

```
[linux@io] echo hello >> file2.txt
[linux@io] cat file2.txt
```

```
hello
```

```
[linux@io] ./opfiles
```

```
file.txt open in read mode
```

```
file2.txt open/created in write mode (truncate to 0)
```

```
[linux@io] cat file2.txt
```

```
[linux@io]
```

The `O_TRUNC` flag causes `open` to truncate the file to a size of 0. In this way, the previous contents of the file are overwritten.

I.2.1 The `fdopen` and `fileno` functions

The `fdopen`²³ function associates a stream with an existing file descriptor passed as its first argument. It returns a stream that refers to the file corresponding to that descriptor.

```
#include <stdio.h>
```

```
FILE *fdopen(int fd, const char *mode);
```

Returns: a pointer to the file

The `mode` argument must specify a stream mode that is compatible with the mode of the file descriptor `fd`. The file position indicator (cursor) of the stream is set to the same position as that of the underlying file descriptor `fd`.

All *file streams* have an underlying file descriptor corresponding to the open file. The `fileno`²⁴ function retrieves the file descriptor associated with a given stream and returns it.

```
#include <stdio.h>
```

```
int fileno(FILE *stream);
```

Returns: the file descriptor

The `fileno.c` program demonstrates an example of how to use the `fileno` function.

```
1 #include <stdio.h>
```

```
2 #include <unistd.h>
```

```
3 int main (int argc, char *argv[])
```

```
4 {
```

```
5     if (argc != 2) {
```

²³ See the `fopen(3)` manual page

²⁴ See the `fileno(3)` manual page

```

6         printf ("\nUsage %s <filename>", argv[0]);
7         return 1;
8     }

9     FILE *fp;
10    int fd;
11    char buffer[32];
12    char *filename = argv[1];

13    if ((fp = fopen(filename, "r") ) == NULL)
14        perror ("fopen");

15    if ((fd = fileno(fp)) == -1)
16        perror ("fileno");

17    printf("\nReading from file pointer..\n");
18    while (fgets(buffer, 32, fp))
19        fputs(buffer, stdout);

20    printf("\n\nReading from file descriptor..");
21    fseek(fp, 0, SEEK_SET);
22    read(fd, buffer, sizeof(buffer));
23    printf("\n%s", buffer);
24    fclose(fp)
25    return 0;
26 }

```

fileno.c

We use `fseek`²⁵ to move the file cursor back to the beginning of the file.

The goal of the program is to determine whether we can read from the file descriptor obtained using `fileno` after already reading the file through the file pointer `fp` (returned by `fopen`). Before performing the second `read`, we must reset the file position to the beginning of the file; otherwise, the cursor will be located somewhere inside the file rather than at its start. This is because the `while` loop has already read the entire content of the file, so after line 22 the file position indicator is no longer at the beginning.

```
[linux@io] echo hello world > newfile.txt
```

```
[linux@io] echo Linux programming >> newfile.txt
```

```
[linux@io] ./fileno ./newfile.txt
```

Reading from file pointer..

```
hello world
```

```
Linux programming
```

25 See the `fseek(3)` manual page

Reading from file descriptor..

hello world

Linux programming

Both `fgets` and `read` have correctly read the file. Let's see what happens if we comment out the call to `fseek`.

```
21 // fseek(fp, 0, SEEK_SET);
```

```
[linux@io] ./fileno ./newfile.txt
```

Reading from file pointer..

hello world

Linux programming

Reading from file descriptor..

Linux programming

`read` has read the file starting from the current position of the file cursor. This position is determined by the result of the last `fgets` call; therefore, the cursor is not at the beginning of the file. If we change the buffer size (and thus the number of bytes `fgets` reads), the cursor will end up at a different position in the file.

The program `fdopen.c` shows an example of how to use `fdopen`.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <fcntl.h>

4 int main (int argc, char *argv[]){
5     if (argc != 2) {
6         printf ("\nUsage %s <filename>", argv[0]);
7         return 1;
8     }
9     int fd;
10    FILE *fp;
11    char buffer[32];
12    char *filename = argv[1];

13    if ((fd = open(filename, O_RDONLY) ) == -1)
14        perror ("open");
15    if ((fp = fdopen(fd, "rw")) == NULL)
16        perror ("fdopen");

17    printf("\nReading from file descriptor..");
18    read(fd, buffer, sizeof(buffer));
19    printf("\n%s",buffer);
20    printf("\n\nReading from the file pointer..\n");
21    fseek(fp, 0, SEEK_SET);
22    while (fgets(buffer, 32, fp))
```

```
23         fputs(buffer, stdout);  
24     fclose(fp);  
25     return 0;  
26 }
```

fdopen.c

The `fdopen` call associates the file pointer `fp` with the file descriptor `fd`.

```
[linux@io] ./fdopen ./newfile.txt
```

Reading from file descriptor..

hello world

Linux programming

Reading from the file pointer..

hello world

Linux programming

In both programs we close the file pointer `fp` using `fclose`. Nothing changes if we close the file descriptor `fd` with `close`.

```
close(fd);
```

The file pointer `fp` refers to the file descriptor `fd`, so closing `fp` is equivalent to closing `fd`.

Since both refer to the same underlying file descriptor, any subsequent read operation, whether performed through the file pointer (e.g., `fgets`) or directly via the file descriptor (e.g., `read`), will continue from the current cursor position left by the previous read.

The book is available on Google Play

<https://play.google.com/store/books/details?id=cZzxEQAAQBAJ>

LINUX

SYSTEM PROGRAMMING

— THE DEFINITIVE GUIDE —

THE COMPLETE RESOURCE FOR LINUX SYSTEM PROGRAMMERS

Linux is more than an operating system—it's a robust, flexible, and elegant platform for building powerful software. This definitive guide provides a deep, hands-on exploration of Linux system programming, covering everything from file I/O and filesystems to processes, memory, IPC, and the runtime environment.

Whether you're writing utilities, daemons, servers, or performance-critical applications, this book gives you the knowledge and techniques to build secure, efficient, and reliable software that runs seamlessly on Linux.



COMPLETE & COMPREHENSIVE

Covers all essential areas of Linux system programming in depth.



PRACTICAL & HANDS-ON

Real-world examples, best practices, and production-ready techniques.



EXPERT LEVEL

Designed for developers who want to go beyond the basics and master Linux internals.



UP-TO-DATE & RELIABLE

Written with modern Linux systems in mind and aligned with current standards.

WHAT YOU'LL MASTER



FILE I/O

- File descriptors & operations
- Pipes, FIFOs & streams
- Buffered & unbuffered I/O
- Memory streams & zero-copy I/O
- Asynchronous I/O & io_uring
- File locking & synchronization
- Error handling & resilience



FILESYSTEM

- Files, directories & metadata
- Permissions, ownership & ACLs
- Links, symlinks & renaming
- Extended attributes
- Device files & special files
- Mounting filesystems
- Inotify & fanotify



RUNTIME ENVIRONMENT

- Time, clocks & timers
- Environment variables
- User & group databases
- Locale & internationalization
- Login, sessions & PAM
- Process credentials & capabilities



MEMORY

- Virtual memory & address space
- Memory mapping (mmap)
- Shared & anonymous memory
- Memory protection
- Memory locking
- Huge pages & tuning



PROCESSES

- Process creation & execution
- Process control & signals
- Threads & concurrency
- Scheduling & priorities
- Synchronization primitives
- Resource limits



IPC AND TERMINALS

- Pipes, message queues
- Shared memory
- Semaphores
- Sockets & domain sockets
- TTYS, terminals & pty's
- I/O multiplexing (poll, epoll)

POWERFUL KNOWLEDGE. REAL-WORLD CODE.

```
01 int main(int argc, char *argv[]) {
02     int fd = open("data.txt", O_RDONLY);
03     if (fd == -1) {
04         perror("open");
05         return 1;
06     }
07
08     char buf[4096];
09     ssize_t n;
10     while ((n = read(fd, buf, sizeof(buf))) > 0)
11         if (write(STDOUT_FILENO, buf, n) != n)
12             perror("write");
13     break;
14 }
15
16 close(fd);
17
18 int pipefd[2];
19 if (pipe(pipefd) == -1) exit(1);
20 pid_t pid = fork();
21 if (pid == 0) { // child
22     dup2(pipefd[1], STDOUT_FILENO);
23     close(pipefd[0]); close(pipefd[1]);
24     execlp("/bin/ls", "ls", "-l", NULL);
25     perror("execlp");
26     _exit(1);
27 }
28 close(pipefd[1]);
29 while ((n = read(pipefd[0], buf, sizeof(buf))) > 0)
30     write(STDOUT_FILENO, buf, n);
31 close(pipefd[0]);
32 wait(NULL);
33
34 struct stat st;
35 if (stat(".", &st) == -1) perror("stat");
36 else printf("size: %lld bytes\n", (long long)st.st_size);
37 return 0;
38 }
```



Master the Linux internals.
Write better code.
Build the systems that power the world.